

Module: 4	CONTROL LOGIC DESIGN: Control organization, Hardwired control, microprogram control, control of processor unit, micro-program sequencer, micro-programmed CPU organization, horizontal and vertical micro instructions,
------------------	--

CONTROL LOGIC DESIGN

The process of logic design is a complex undertaking. The binary information found in a digital system is stored in processor or memory registers, and it can be either data or control information.

The logic design of a digital system is a process for deriving the digital circuits that perform data processing and the digital circuits that provide control signals. The timing for all registers in a synchronous digital system is controlled by a master-clock generator. The clock pulses are applied to all flip-flops and registers in the system, including the flip-flops and registers in the control unit.

Two representations which are helpful in the design of systems that need a control are timing diagrams and flowcharts. A **timing diagram** clarifies the timing sequence and other relationships among the various control signals in the system. A **flowchart** is a convenient way to specify the sequence of procedural steps and decision paths for an algorithm.

The design of control logic cannot be separated from the algorithmic development necessary for solving a design problem. Moreover, the control logic is directly related to the data-processor part of the system that it controls.

CONTROL ORGANIZATION

Once a control sequence has been established, the sequential system that implements the control operations must be designed. Since the control is a sequential circuit, it can be designed by a sequential logic procedure.

Disadvantages of sequential control logic are

- Large number of states
- Excessive number of flip-flops and gates
- Design methods uses state and excitation tables but in practice they are cumbersome

Goal of control logic design should be development of a circuit that implements the desired control sequence in a logical and straightforward manner. Designers used specialized methods for control logic design which is considered as the extension of the classical sequential logic method combined with register transfer method.

We consider four methods of control organization

- One flip-flop per state methods
- Sequence register and decoder method
- PLA control
- Micro-program control

The first two methods result in a circuit that must use SSI and MSI circuits for the implementation. A control unit implemented with SSI and MSI devices is said to be a hard-wired control. If any alterations or modifications are needed, the circuits must be rewired to fulfill the new requirements.

The PLA or micro-program control which uses an LSI device such as a programmable logic array or a read-only memory. Any alterations or modifications in a micro-program control can be easily achieved without wiring changes by removing the ROM from its socket and inserting another ROM programmed to fulfill the new specifications.

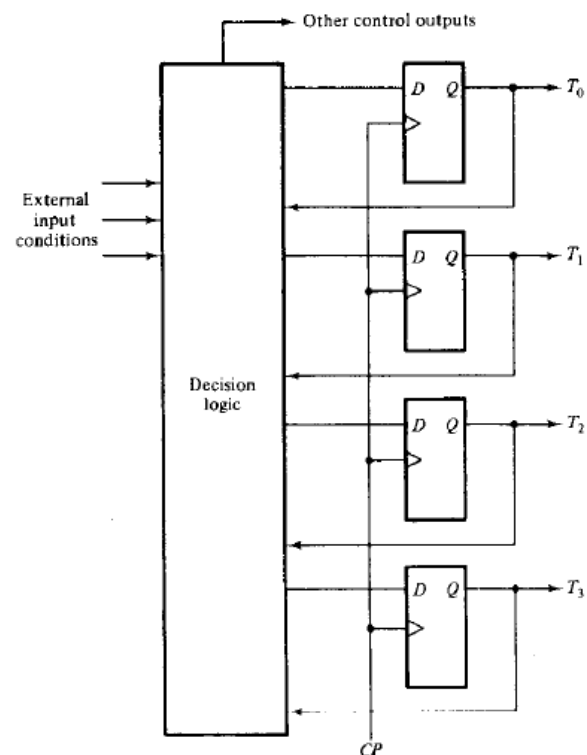
One flip-flop per state methods

This method uses one flip-flop per state in the control sequential circuit. Only one flip-flop is set at any particular time: all others are cleared. A single bit is made to propagate from one flip-flop to the other under the control of decision logic. In such an array, each flip-flop represents a state and is activated only when the control bit is transferred to it.

In this method, maximum numbers of flip-flops were used. Example: A sequential circuit with 12 states requires a minimum of four flip-flops because $2^3 < 12 < 2^4$. Control circuit uses 12 flip-flops, one for each state

The **advantage** of this method is the simplicity with which it can be designed. This type of controller can be designed by inspection from the state diagram that describes the control sequence. This also offers other advantages like savings in design effort, an increase in operational simplicity, and a potential decrease in the combinational circuits required to implement the complete sequential circuit. The **disadvantage** is that this method would increase system cost since more flip-flops are used.

Figure shows the configuration of a four-state sequential control logic that uses four D-type flip-flops: one flip-flop per state T_i , $i = 0, 1, 2, 3$.



At any given time interval between 2 clock pulses, only one flip-flop is equal to 1; all others are 0. The transition from the present state to next state is a function of the present T_i that is a 1 and certain input conditions.

The next state is manifested when the previous flip-flop is cleared and a new one is set. Each of the flip-flops is connected to the data processing section of the digital system to initiate certain micro-operations. The control outputs are a function of the T 's and external inputs. These outputs may also initiate micro-operations

If control circuit does not need external inputs for its sequencing, the circuit reduces to straight shift register with a single bit shifted from one position to the next. If control sequence must repeated over and over again the control reduces to ring counter.

Ring counter is a shift register with the output of last flip-flop connected to the input of the first flip-flop. In a ring counter single bit continuously shifts from one position to the next in a circular manner.

For this reason, this method is also called ring counter controller

Sequence Register and Decoder Method

This method uses a register to sequence the control states. The register is decoded to provide one output for each state. For n flip-flops in the sequence register, the circuit will have 2^n states and the decoder will have 2^n outputs.

For example, a 4-bit register can be in any one of 16 states. A 4×16 decoder will have 16 outputs, one for each state of the register. Both the sequence register and decoder are MS1 devices.

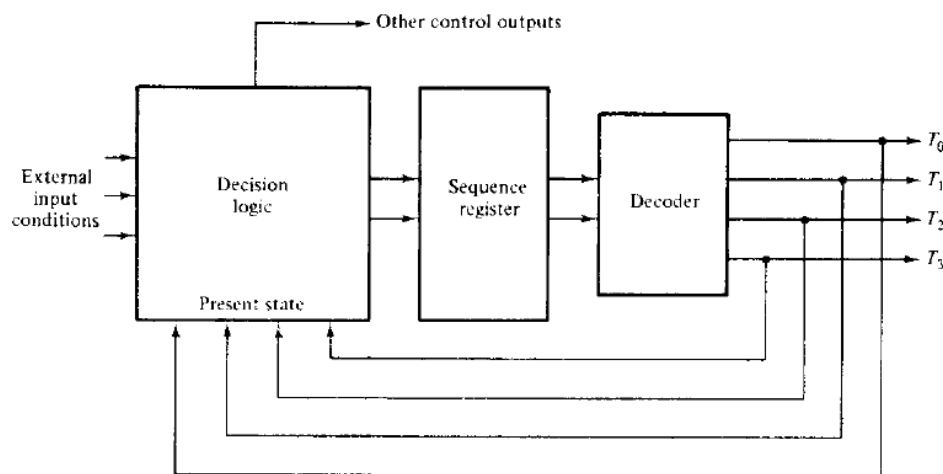
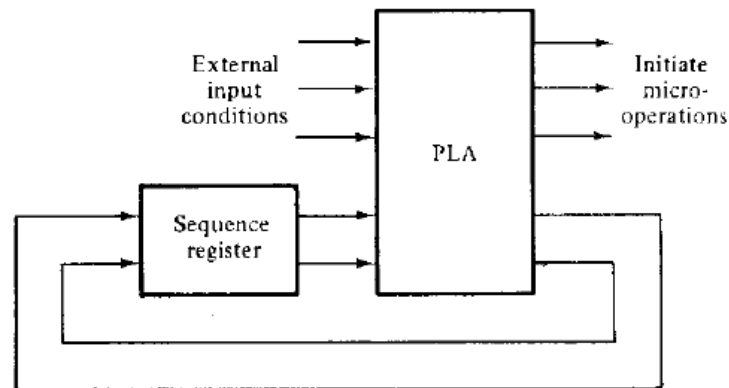


Figure shows the configuration of a four-state sequential control logic. The sequence register has two flip-flops and the decoder establishes separate outputs for each state in the register. The transition to the next state in the sequence register is a function of the present state and the external input conditions.

If the control circuit does not need external inputs the sequence register reduces to a counter that continuously sequence through the four states so called counter decoder method

PLA control

The external sequence register establishes the present state of the control circuit. The PLA outputs determine which micro-operations should be initiated depending on the external input conditions and the present state of the sequence register. At the same time other PLA outputs determine the next state of the sequence register.



The sequence register is external to the PLA if the unit implements only combinational circuits. Some PLAs include not only gates but also flip-flops within the unit. This implements a sequential circuit by specifying the links that must be connected to the flip-flops in manner that the gate links are specified

Micro-program Control

The purpose of control unit is to initiate a series of sequential steps of micro-operations. At any given time certain operations are to be initiated while all others remain idle. The control variable at any given time can be represented by a string of 1's and 0's called **control word**. The control words can be programmed to initiate the various components in the system in an organized manner.

A control unit whose control variables are stored in a memory called a **micro-programmed control** unit. Each control word of memory is called **Microinstruction** and Sequence of microinstructions is called **Micro-program**.

Control memory is usually ROM since an alteration of micro-program is seldom needed. The use of micro-program involves placing all control variables in words of the ROM for use by the control unit through successive read operations. The content of the word in the ROM at a given address specifies the micro-operations for the system.

Dynamic micro-programming permits a micro-program to be loaded initially from the computer console or from an auxiliary memory such as magnetic disk. Writable control

memory(WCM) can be used for writing but used mostly for reading. A ROM, PLA or WCM when used in a control unit is referred as a control memory.

Control memory address register specifies the control word read from control memory. The ROM operates as a combinational circuit with address value as the input and the corresponding word as the output. The content of the specified word remains on the output wires as long as the address value remains in the address register.

If the address registers change while the ROM word is still in use then the word out of the ROM should be transferred to a buffer register. If the change in address and ROM word can occur simultaneously no buffer register is needed. The word read from memory represents a microinstruction. The microinstruction specifies one or more micro-operations for the components of the system.

Once these operations are executed, the control unit must determine its next address. The location of the next microinstruction may be next one in the sequence or it may locate somewhere else in the control memory. Some bits of the microinstruction control the generation of the address for the next microinstruction.

The next address may be a function of external input conditions. The next address is computed in the next address generator circuit and then transferred into the control address register to read the next microinstruction.

HARDWIRED CONTROL

The control hardware can be viewed as a state machine that changes from one state to another in every clock cycle, depending on the contents of the instruction register, the condition codes and the external inputs. The outputs of the state machine are the control signals. The sequence of the operation carried out by this machine is determined by the wiring of the logic elements and hence named as “hardwired”.

Control logic derived in this section is a hardwired control of the one flip-flop per state method. The design of hardwired control is carried out in 5 consecutive steps

1. The problem is stated
2. An initial equipment configuration is assumed
3. An algorithm is formulated
4. The data processor part is specified
5. The control logic is designed

Statement of the problem

The problem here is to implement with hardware the addition and subtraction of two fixed-point binary numbers represented in sign-magnitude form. Complement arithmetic may be used, provided the final result is in sign-magnitude form.

The addition of two numbers stored in registers of finite length may result in a sum that exceeds the storage capacity of the register by one bit. The extra bit is said to cause an overflow. The circuit must provide a flip-flop for storing a possible overflow bit.

Equipment Configuration

The two signed binary numbers to be added or subtracted contain n bits. The magnitudes of the numbers contain $k = n - 1$ bits and are stored in registers A and B. The sign bits are stored in flip-flops A_s and B_s .

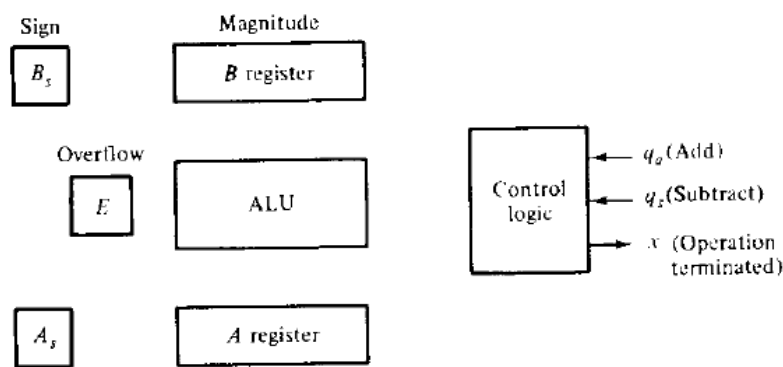


Figure shows the registers and associated equipment. The ALU performs the arithmetic operations and the 1-bit register E serves as the overflow flip-flop. The output carry from the ALU is transferred to E .

It is assumed that the two numbers and their signs have been transferred to their respective registers and that the result of the operation is to be available in registers A and A_s . Two input signals in the control specify the add (q_a) and subtract (q_s) operations. Output variable x indicates the end of the operation.

The control logic communicates with the outside environment through the input and output variables. Control recognizes input signal q_a or q_s and provides the required operation. Upon completion of the operation, control informs the external environment with output x that the sum or difference is in registers A and A_s , and that the overflow bit is in E .

Derivation of the Algorithm

Designate the magnitude of the two numbers by A and B . When the numbers are added or subtracted algebraically, there are eight different conditions to consider, depending on the sign of the numbers and the operation performed. The eight conditions may be expressed in a compact form as follows:

$$(\pm A) \pm (\pm B)$$

If the arithmetic operation specified is subtraction, we change the sign of B and add. This is evident from the relations:

$$(\pm A) - (+B) = (\pm A) + (-B)$$

$$(\pm A) - (-B) = (\pm A) + (+B)$$

This reduces the number of possible conditions to four, namely:

$$(\pm A) + (\pm B)$$

The four possible combination are

When the signs of A and B are the same: $(+A) + (+B) = +(A + B)$

$$(-A) + (-B) = -(A + B)$$

When the signs of A and B are not the same

$$(+A) + (-B) = +(A - B) \quad [\text{if}(A > B)]$$

$$-(B - A) \quad [\text{if}(B > A)]$$

$$(-A) + (+B) = -(A - B) \quad [\text{if}(A > B)]$$

$$+(B - A) \quad [\text{if}(B > A)]$$

The flowchart shows how we can implement sign-magnitude addition and subtraction with the equipment as shown in previous diagram.

For more details about flowchart, refer text book "Digital Logic and Computer Design" by M. Morris Mano; Page 418.

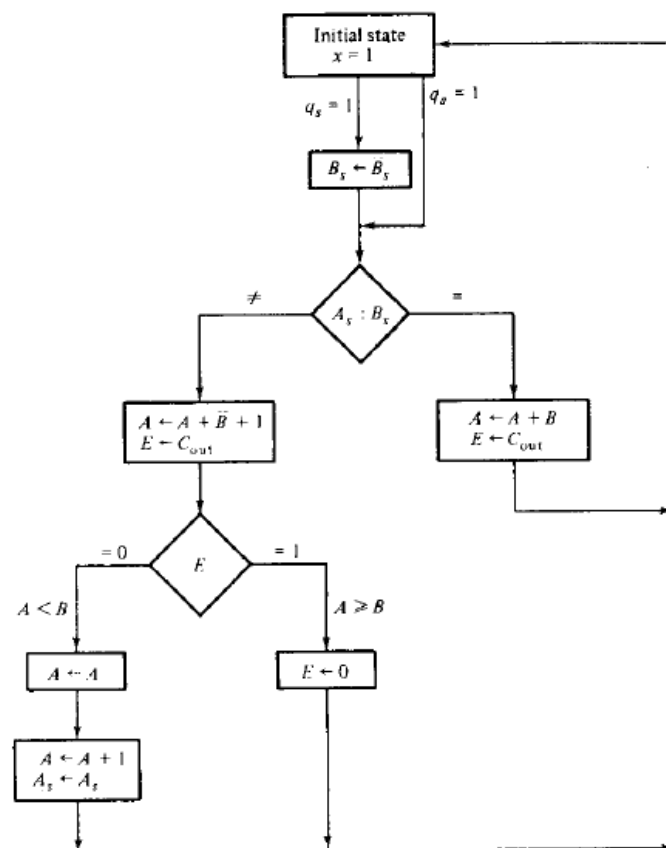


Figure (a) shows the data-processor with the required control variables. This ALU has four selection variables, as shown in the diagram. The variable S loads the output of the ALU into register A and also the output carry into E. Variables y , z , and w complement B, and A, and clear E, respectively.

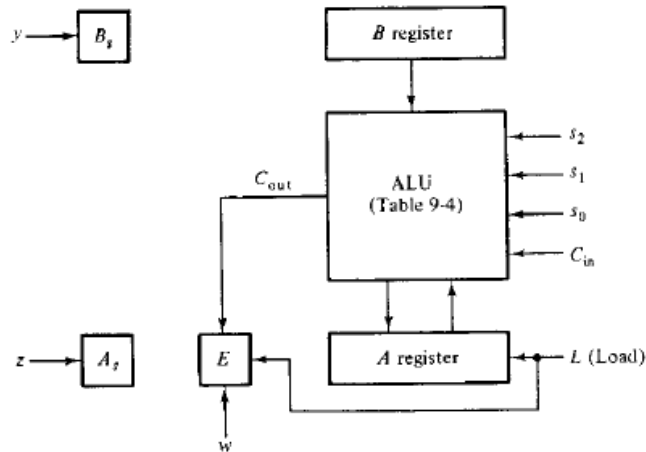


Figure a. Data Processor Register and ALU

The block diagram of the control logic is shown in figure (b). The control receives five inputs: two from the external environment and three from the data-processor. To simplify the design, we define a new variable

$$S = A_s \oplus B_s$$

This variable gives the result of the comparison between the two sign bits. The exclusive-OR operation is equal to 1 if the two signs are not the same, and it is equal to 0 if the signs are both positive or both negative. The control provides an output x for the external circuit. It also selects the operations in the ALU through the four selection variables S_2 , S_1 , S_0 , and C_{in} . The other four outputs go to registers in the data-processor as specified in the diagram.

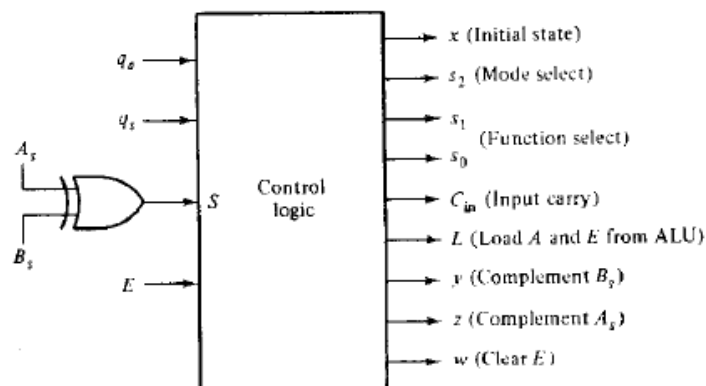
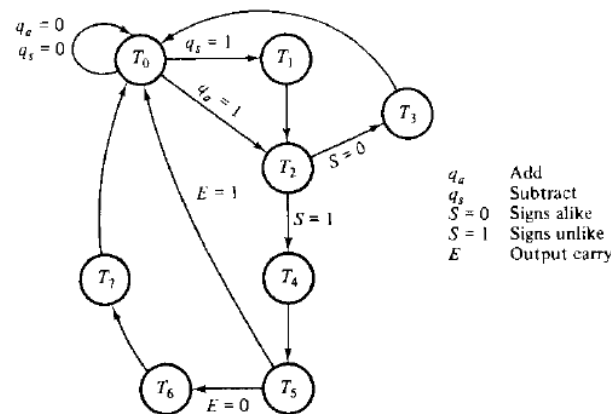


Figure b. Control Block Diagram

Control State Diagram

The design of a hard-wired control is a sequential-logic problem. As such, it may be convenient to formulate the state diagram of the sequential control. The function boxes in a flowchart may be considered as states of the sequential circuit, and the decision boxes as next-state conditions.

The micro-operations that must be executed at a given state are specified within the function box. The conditions for the next state transition are specified inside the decision box or in the directed lines between two function boxes. Consequently, different designers may produce different state diagrams for the same flowchart, and each may be a correct representation of the system. The control state diagram and the corresponding register-transfer operations are derived in below figure.



We start by assigning an initial state, T_0 , to the sequential controller. We then determine the transition to other states T_1 , T_2 , T_3 , and so on. For each state, we determine the micro-operations that must be initiated by the control circuit. The figure below shows the sequence of register transfers.

	Control outputs								
	x	s_2	s_1	s_0	C_{in}	L	y	z	w
T_0 : Initial state $x = 1$	1	0	0	0	0	0	0	0	0
T_1 : $B_x \leftarrow \bar{B}_x$	0	0	0	0	0	0	1	0	0
T_2 : nothing	0	0	0	0	0	0	0	0	0
T_3 : $A \leftarrow A + B, E \leftarrow C_{out}$	0	0	0	1	0	1	0	0	0
T_4 : $A \leftarrow A + \bar{B} + 1, E \leftarrow C_{out}$	0	0	1	0	1	1	0	0	0
T_5 : $E \leftarrow 0$	0	0	0	0	0	0	0	0	1
T_6 : $A \leftarrow \bar{A}$	0	1	1	1	0	1	0	0	0
T_7 : $A \leftarrow A + 1, A_y \leftarrow \bar{A}_y$	0	0	0	0	1	1	0	1	0

Design of Hardwired Control

The control can be designed using the classical sequential-logic procedure. This procedure requires a state table with eight states, four inputs, and nine outputs. The sequential circuit to be derived from such a state table will not be easy to obtain because of the large number of variables.

The circuit obtained by using this method may have a minimum number of gates, but it will have an irregular pattern and will be difficult to analyze if a malfunction occurs. These difficulties are removed if the control is designed by the one flip-flop per state method.

A control organization that uses one flip-flop per state has the convenient characteristic that the circuit can be derived directly from the state diagram by inspection. No state or excitation tables are needed if D flip-flops are employed.

Remember that the next state of a D flip-flop is a function of the D input and is independent of the present state. Since the method requires one flip-flop for each state, we choose eight D flip-flops and label their outputs $T_0, T_1, T_2, \dots, T_7$. The condition for setting a given flip-flop is specified in the state diagram.

For example, flip-flop T_2 is set with the next clock pulse if $T_1 = 1$ or if $T_0 = 1$ and $q_a = 1$. This condition can be defined with the Boolean function:

$$DT_2 = q_a T_0 + T_1$$

where DT_2 designates the D input of flip-flop T_2 . If there is more than one directed line going into a state, all conditions must be ORed. Using this procedure for the other flip-flops, we obtain the input functions given in table below.

Flip-flop input functions	Boolean functions for output control
$DT_0 = q_a q_s' T_0 + T_3 + ET_5 + T_7$	$x = T_0$
$DT_1 = q_s T_0$	$s_2 = T_6$
$DT_2 = q_a T_0 + T_1$	$s_1 = T_4 + T_6$
$DT_3 = S' T_2$	$s_0 = T_3 + T_5$
$DT_4 = ST_2$	$C_{in} = T_4 + T_7$
$DT_5 = T_4$	$L = T_3 + T_4 + T_6 + T_7$
$DT_6 = E' T_5$	$y = T_1$
$DT_7 = T_6$	$z = T_7$
	$w = T_5$

Initially, flip-flop T_0 is set and all others are cleared. At any given time, only one D input is in the 1 state while all others are maintained at 0. The next clock pulse sets the flip-flop whose D input is 1 and clears all others.

The circuit for the control logic is not drawn but can be easily obtained from the Boolean functions in table. The circuit can be constructed with eight D flip-flops, seven AND gates, six OR gates, and four inverters. Note that five control outputs are taken directly from the flip-flop outputs.

MICRO-PROGRAM CONTROL

In a micro-program control, the control variables that initiate micro-operations are stored in memory. The control memory is usually a ROM, since the control sequence is permanent and needs no alteration. The control variables stored in memory are read one at a time to initiate the sequence of micro-operations for the system.

Each micro operation is associated with a specific set of control lines which, when activated, causes that micro operation to take place. Since the number of instructions and control lines is often in the hundreds, the complexity of hardwired control unit is very high.

The hardwired control unit is relatively inflexible because it is difficult to change the design, if one wishes to correct design error or modify the instruction set

The micro programmed control unit consists of following components.

Control memory:

In micro programmed control, the micro programs for all instructions are stored in the control memory (CM). The control signals to be activated at any time are specified by a microinstruction, which is fetched from Control memory (CM).

Control address register

The control address register holds the address of the next microinstruction to be read. When address is available in control address register, the sequencer issues READ command to the control memory.

Microinstruction register

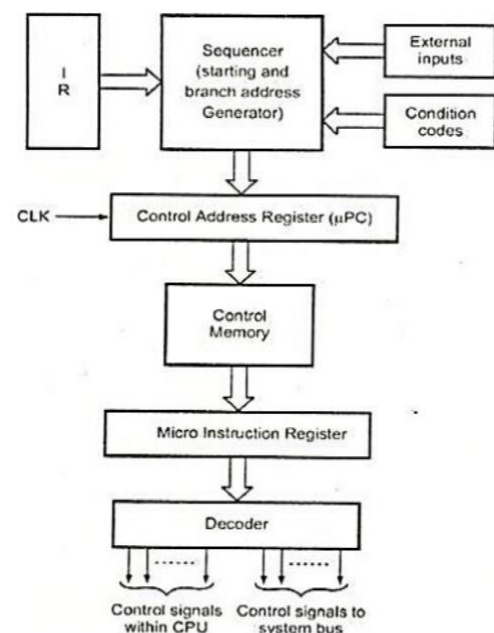
After issue of READ command, the word from the addressed location is read into the microinstruction register. Now the content of the micro instruction register generates control signals and next address information for the sequencer.

Micro-program sequencer

A sequence of one or more micro operations designed to control specific operation, such as addition, multiplication is called a micro program. The sequencer loads a new address into the control address register based on the next address information.

Advantages of Micro programmed control

- It simplifies the design of control unit. Thus it is both, cheaper and less error prone implement.
- Control functions are implemented in software rather than hardware.
- The design process is orderly and systematic



- More flexible, can be changed to accommodate new system specifications or to correct the design errors quickly and cheaply.
- Complex function such as floating point arithmetic can be realized efficiently.

Disadvantages

- A micro programmed control unit is somewhat slower than the hardwired control unit, because time is required to access the microinstructions from CM
- The flexibility is achieved at some extra hardware cost due to the control memory and its access circuitry.

Microinstructions

The words stored in a control memory are microinstructions, and each microinstruction specifies one or more micro-operations for the components in the system.

Example: Control Sequence for instruction Add (R3), R1:

1. PCout, MARin, Read, Select4, Add, Zin
2. Zout, PCin, Yin, Wait for the MFC
3. MDROUT, IRin
4. R3out, MARin, Read
5. R1out, Yin, Wait for MFC
6. MDROUT, Select Y, Add, Zin
7. Zout, R1in, End

The microinstruction for the above control sequence can be expressed as follows.

Micro instruction	..	PC _{in}	PC _{out}	MAR _{in}	Read	MDROUT	IR _{in}	Y _{in}	Select	Add	Z _{in}	Z _{out}	R1 _{out}	R1 _{in}	R3 _{out}	VMFC	End	..
1		0	1	1	1	0	0	0	1	1	1	0	0	0	0	0	0	
2		1	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	
3		0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	
4		0	0	1	1	0	0	0	0	0	0	0	0	0	1	0	0	
5		0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	0	
6		0	0	0	0	1	0	0	0	1	1	0	0	0	0	0	0	
7		0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	1	

Once these micro-operations are executed, the control unit must determine its next address. Therefore, a few bits of the microinstruction are used to control the generation of the address for the next microinstruction. Thus, a microinstruction contains bits for initiating micro-operations and bits that determine the next address for the control memory itself.

Techniques of grouping of Control Signals

The grouping of control signal can be done either by using technique called

- Vertical organization,

- Horizontal organization.

Horizontal Micro-Instructions

The scheme of micro-instruction by assigning one bit position to each control signal is called **horizontal micro-instructions**.

Example: 011101001101001110

In a horizontal microinstruction every bit in the control field attaches to a controller. Horizontal microinstructions represent several micro-operations that are executed at the same time. However, in extreme cases, each horizontal microinstruction controls all the hardware resources of the system.

Vertical Micro-Instructions

We can reduce the length of the horizontal micro-instruction so easily by implementing another method known as vertical micro-instructions. In this case, Most signals are not needed simultaneously and many others are mutually exclusive

Example:

F1 (4 bits)	F2 (3 bits)	F3 (3 bits)	AS3 AS2 AS1	Read	Write	Carry-in	WMFC	End
0000: No action	000: No action	000: No action	000: ADD					
0001: PCout	001: PCin	001: MARin	001: INC					
0010: MDRout	010: IRin	010: MDRin	010: SUB					
0011: Zout	011: Zin	011: TEMPin	011: DEC					
0100: R0out	100: R0in	100: Yin	100: AND					
0101: R1out	101: R1in		101: OR					
0110: R2out	110: R2in		110: XOR					
0111: R3out	111: R3in		111: NOT					
1000: IRout								
1001: TEMPout								

In a vertical microinstruction, a code is used for each action to be performed and the decoder translates this code into individual control signals. The vertical microinstruction resembles the conventional machine language format comprising one operation and a few operands. As opposed to horizontal microinstructions, the vertical microinstruction represents single micro-operations.

Advantage

- Fewer bits are required in the microinstruction.

Disadvantage

- Vertical approach results in slower operations speed.

Comparison between Horizontal and Vertical Organization

Horizontal	Vertical
------------	----------

Long formats.	Short formats
Ability to express a high degree of parallelism.	Limited ability to express parallel micro operations.
Little encoding of the control information.	Considerable encoding of the control information
Useful when higher operating speed is desired.	Slower operating speeds.

Micro-program Sequencer

Micro program sequencer is a control unit which does the tasks of Micro-program sequencing. There are two important factors must be considered while designing the micro program sequencer.

- The size of the microinstruction
- The address generation time

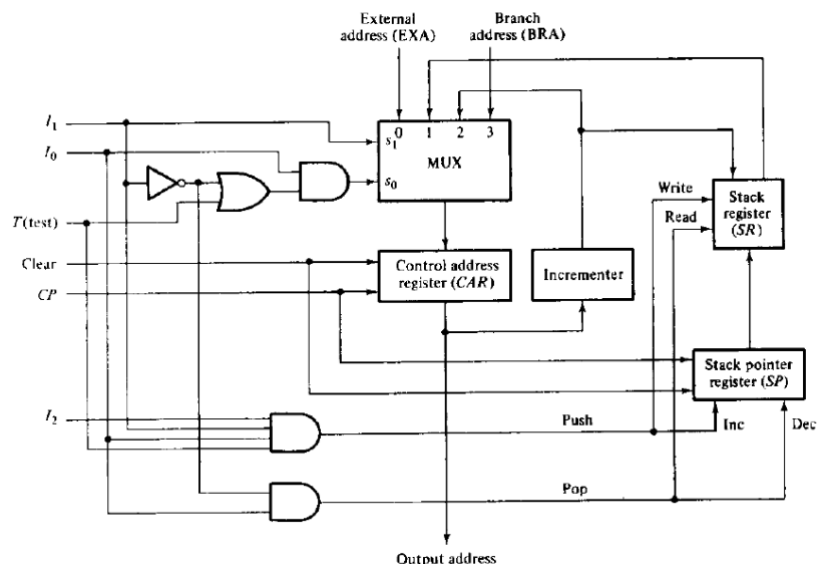
Micro-program sequencer is attached to the control memory. It inspects certain bits in the microinstruction to determine the next address for control memory. A typical sequencer has the following address sequencing capabilities.

1. Increments the present address of control memory
2. Branches to an address which will be specified in the bits of microinstruction
3. Branches to a given address if a specified status bit is equal to 1.
4. Transfers control to a new address as specified by an external source
5. Has a facility for subroutines calls and returns.

The block diagram of Micro-program Sequencer is shown in the below figure. It consists of a multiplexer that selects an address from four sources and routes it into a control address register (CAR).

The output from CAR provides the address for control memory. The contents of CAR are incremented and applied to the multiplexer and to the stack register file.

The register selected in the stack is determined by stack pointer. Inputs (I_0 - I_2) specify the operation for the sequencer and input T is the test point for a status bit. Initially the address register is cleared to zero and clock pulse synchronizes the loading into registers.



Function table							
I_2	I_1	I_0	T	s_1	s_0	Operation	Comments
X	0	0	X	0	0	$CAR \leftarrow EXA$	Transfer external address
X	0	1	X	0	1	$CAR \leftarrow SR$	Transfer from register stack
X	1	0	X	1	0	$CAR \leftarrow CAR + 1$	Increment address
0	1	1	0	1	0	$CAR \leftarrow CAR + 1$	Increment address
0	1	1	1	1	1	$CAR \leftarrow BRA$	Transfer branch address
1	1	1	0	1	0	$CAR \leftarrow CAR + 1$	Increment address
1	1	1	1	1	1	$CAR \leftarrow BRA, SR \leftarrow CAR + 1$	Branch to subroutine

For more details about this, refer text book "Digital Logic and Computer Design" by M. Morris Mano; Page 446 - 449

Micro-programmed CPU Organization

Digital computer consists of: Central Processing Unit(CPU), Memory unit and Input-output devices. CPU can be divided into 2 distinct and interactive sections namely *processing section*: useful device for constructing the processor section of a CPU and *control section*: controlling the entire units of computer, micro-program sequencer: constructing a micro-program control of CPU

Micro-programmed computer

A computer CPU uses the micro-program sequencer. Micro-programmed computer consists of

1. Memory unit: Stores instructions and data supplied by the user through an input device
2. Two processor unit: *Data processor*: Manipulates data & *Address processor*: Manipulates the address information received from memory
3. A Micro-program sequencer
4. A control memory
5. Other digital functions

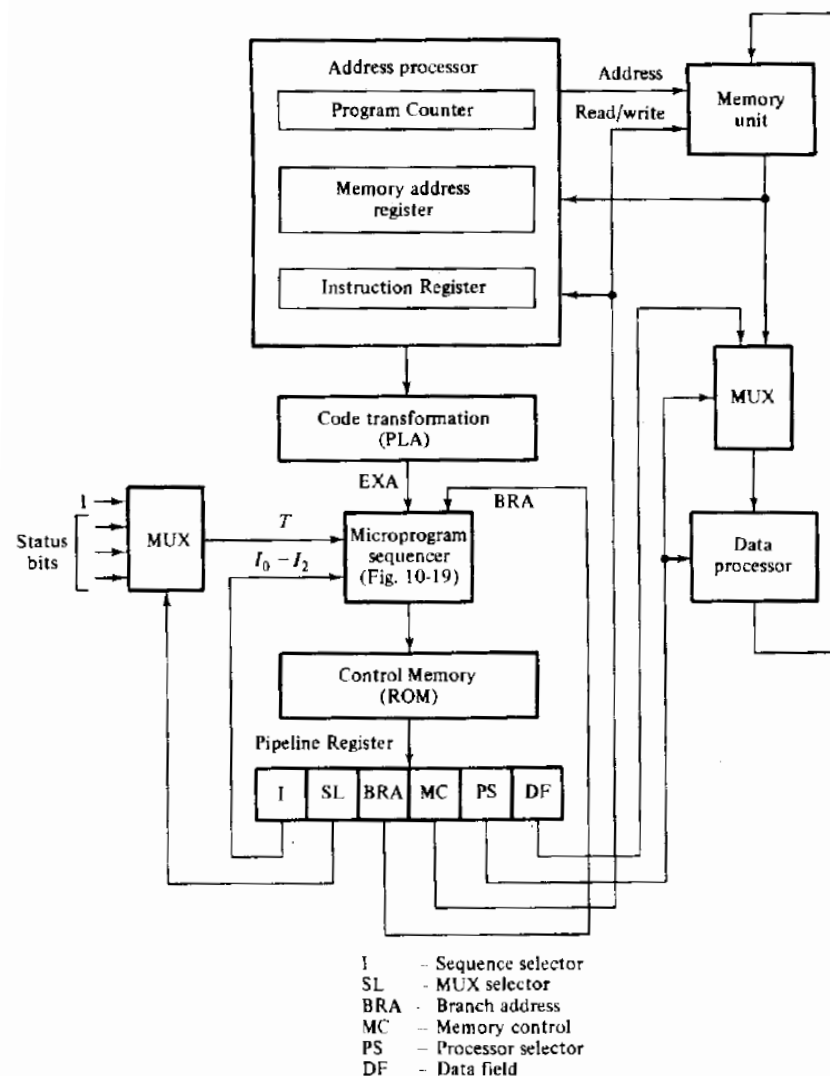
An instruction extracted from memory unit during fetch cycle goes into instruction register. Code transformation constitutes a mapping function that is needed to convert the operation code bits of an instruction into a starting address for the control memory and is implemented with ROM or PLA.

Mapping concept provides flexibility for adding instructions or micro-operations for control memory as need arises. The address generated in code transformation mapping function is applied to the external address (EXA) input of the sequencing.

Micro-program control unit consists of

1. *The sequencer*: Generates next address
2. *A control memory*: Reads the next microinstruction while present microinstruction are being executed in the other units of the CPU and for storing microinstructions
3. *A multiplexer*: Selects one of the many status bits and applies to the T(test) input of the sequencer. One of the input of the multiplexer is always 1 to provide an unconditional branch operation

4. A pipeline register: speed up the control operation, Allows next address to be generated and the output of control memory to change while the control word in pipeline register initiates the micro-operations given by present microinstruction. It's not always necessary. The output of control memory can go directly to the control inputs of the various units in the CPU



Micro-programmed Computer Organization

Microinstruction format contains six fields: First 3 fields(I,SL, BRA) provide information to the sequencer to determine the next address for control memory. [I field (3 bits): Supplies input information for the sequencer, SL field: Selects a status bit for the multiplexer, BRA field: Address field of microinstruction and supplies a branch address (BRA) to the sequencer.]

The next 3 fields (MC, PS, DF) are for controlling micro-operations in the processor and the memory units [Memory control (MC) field: Controls the address processor and the read and

write operations in the memory unit. The processor select(PS) field: Controls the operations in the dataprocessor unit. The data field(DF): Used to introduce constants into the processor]

Output from data field may be used to set up control registers and introduce data inprocessor registers.